

CS599 大作业报告

Mailflow Agent：多 Agent 邮件与日程助理系统

基于 FastAPI、LangGraph、Vue 3、Gmail API 与 Google Calendar API 的本地优先 Agentic AI 应用

项目	内容
课程名称	企业级应用软件设计与开发
项目名称	Mailflow Agent
方向	Agentic AI 原生开发
学号	2025303048
姓名	黎鸿
专业	软件工程
指导教师	戚欣
提交日期	2026 年 6 月 22 日
GitHub 仓库	https://github.com/leehong020/cs599-project

目录

- 目录..... 2
- 一、选题背景与设计思想 4
 - 1.1 问题定义 4
 - 1.2 现有方案不足 4
 - 1.3 项目价值 4
 - 1.4 技术路线 4
- 二、Specs 规范文档 5
 - 2.1 Product Spec 5
 - 2.2 Architecture Spec 5
 - 2.3 API Spec 5
 - 2.4 Specs 与项目文档对应 6
- 三、系统架构与设计 7
 - 3.1 核心架构图 7
 - 3.2 Agent 交互流程 7
 - 3.3 数据流设计 8
- 四、关键实现与代码展示 9
 - 4.1 Agent 核心循环 9
 - 4.2 工具定义与职责隔离 9
 - 4.3 配置文件与 OAuth 9
 - 4.4 AI IDE 使用 10
 - 4.5 关键实现对应文件 10
- 五、测试与评估 11
 - 5.1 自动化测试 11
 - 5.2 Demo 截图与功能验证 11

5.3 Agent 行为评估.....	13
5.4 Demo 结果小结.....	14
5.5 评估结论	14
六、系统升级与扩展.....	15
6.1 可扩展架构	15
6.2 下一阶段计划	15
6.3 AI 能力演进路径.....	15
七、课程总结	16
7.1 个人收获	16
7.2 工程思维转变	16
7.3 对课程的建议	16
参考资料.....	错误!未定义书签。

一、选题背景与设计思想

1.1 问题定义

日常学习和项目协作中，邮件、日程、文件和联系人信息经常分散在不同系统里。用户需要在 Gmail、Google Calendar、聊天记录、课程文档和本地文件之间反复切换，才能完成“读材料、写邮件、约时间、确认结果”这类工作流。Mailflow Agent 选择从这个具体场景切入：让用户用自然语言表达目标，由系统完成上下文读取、草稿生成、字段补全、确认授权和真实工具执行。

1.2 现有方案不足

- 传统邮箱和日历客户端更偏向人工页面操作，不能把多个系统中的上下文自动串联起来。
- 通用聊天助手虽然能生成文本，但缺少 Gmail、Calendar、联系人、文件和持久化状态，难以完成真实执行闭环。
- 普通自动化脚本可以调用 API，但缺少多轮追问、字段来源校验、草稿版本、用户确认和审计记录，安全边界不足。
- 企业级场景中，AI 不能只追求“自动做事”，还必须解释依据、保留可恢复状态，并在高风险写操作前让用户明确确认。

1.3 项目价值

本项目的价值是把大模型能力从单轮文本生成推进到可追踪的软件工作流。系统不仅能写出邮件正文或日程标题，还会把结果保存为本地 Artifact，生成 Proposal 供用户核对，并在 Authorization 通过后调用 Execution Service 写入 Gmail 或 Google Calendar。这样既保留了 Agent 的自然语言交互优势，也把风险控制明确的工程边界内。

1.4 技术路线

- 前端使用 Vue 3 / Vite，提供聊天、Gmail、Calendar 和设置页面。
- 后端使用 FastAPI，统一承接 OAuth、Gmail、Calendar、Assistant、Workflow、Files、Settings 和 Memory API。
- Agent 编排使用 LangGraph / LangChain 风格的状态图，把 Supervisor、Context、Mail、Calendar、Review、Confirmation、Executor、Response 等节点串成可恢复流程。
- 数据层使用 SQLite、SQLAlchemy 和 Alembic，运行时数据放在 data/runtime，仓库只提交代码、配置模板和文档。

二、Specs 规范文档

2.1 Product Spec

产品目标是构建一个本地优先的 AI 邮件和日程助理。MVP 聚焦个人学习和课程提交场景，要求用户可以连接 Google 账号，通过自然语言处理邮件、日程和上传文件，并在真实写操作前进行确认。

模块	规格说明
目标用户	需要统一处理 Gmail、Google Calendar、课程文件、联系人和个人偏好的本地用户。
核心任务	邮件搜索/读取/草稿/确认发送，日程查看/草稿/确认创建，文件上传解析，联系人与署名辅助。
交互原则	用户提出目标，Agent 生成结构化草稿；外部写操作必须显示关键字段并等待确认。
安全要求	OAuth secret、token、API Key、.env、google_oauth.json、data/ 运行数据不得提交到 GitHub。
完成标准	可本地运行，README 配置可复现，测试通过，报告和截图能够证明核心功能闭环。

2.2 Architecture Spec

系统采用“前端交互层 - FastAPI 服务层 - LangGraph 编排层 - 工具执行层 - SQLite 事实来源”的分层架构。模型负责理解与生成，确定性节点负责状态流转、安全门和工具执行，避免让 LLM 直接持有凭据或直接绕过授权。

层次	组件	职责
表现层	Vue 3 / Vite	聊天、Gmail、Calendar、设置、Google 连接状态和 Proposal 确认界面。
服务层	FastAPI	提供 OAuth 回调、Assistant 流式输出、Gmail/Calendar 路由、文件和设置接口。
编排层	LangGraph nodes	维护会话状态、Agent 路由、上下文处理、草稿生成、确认与执行。
工具层	Tool functions / Execution Service	封装 Gmail、Calendar、联系人、署名、文件和记忆工具，统一返回 JSON。
数据层	SQLite / Alembic	保存线程、消息、Work Item、Artifact、Proposal、Authorization、Action Event、设置和记忆。

2.3 API Spec

API 组	典型能力	说明
assistant_graph	聊天、会话、SSE	接收用户消息并驱动 Agent 图运行，返回草稿、追问、确认或执行结果。
auth	Google OAuth	完成授权跳转、回调、连接状态查询和断开连接。
gmail	邮件搜索与发送链路	读取邮件、准备新邮件/回复/转发，并通过执行服务发送。

calendar	日程读取与写入链路	读取事件、忙闲查询、准备日程草稿，并在确认后创建/更新/删除事件。
files/settings/memory	文件、偏好和记忆	支持上传解析、联系人/署名/用户资料，以及长期记忆和导出。

2.4 Specs 与项目文档对应

课程要求中的 Product Spec、Architecture Spec 和 API Spec 没有单独停留在报告里，而是同步落到仓库文档，便于评审者按路径追踪实现范围和设计边界。

规格内容	对应文件	作用
Product Spec	src/project-docs/product_scope.md	说明目标用户、MVP 范围、非目标范围和成功标准。
Architecture Spec	docs/architecture.md / architecture_decisions.md	说明系统分层、Agent 编排、安全闭环和主要 ADR。
API Spec	src/backend/app/api/* / schemas/*	以 FastAPI 路由和 Pydantic Schema 表达接口输入输出。
Safety Spec	safety_rules.md / required_fields.md	约束字段来源、Proposal、Authorization、日志脱敏和凭据保护。
Acceptance Spec	acceptance_matrix.md	把功能、文档、安全和测试要求映射到当前实现。

三、系统架构与设计

3.1 核心架构图

图 1 展示系统从用户交互到外部 Google 服务的主要路径。前端只负责用户可见状态，不保存 OAuth Secret；后端统一读取本地配置和数据库；Agent 编排层决定本轮任务走邮件、日程还是上下文处理；工具层负责真实 API 调用和执行结果记录。

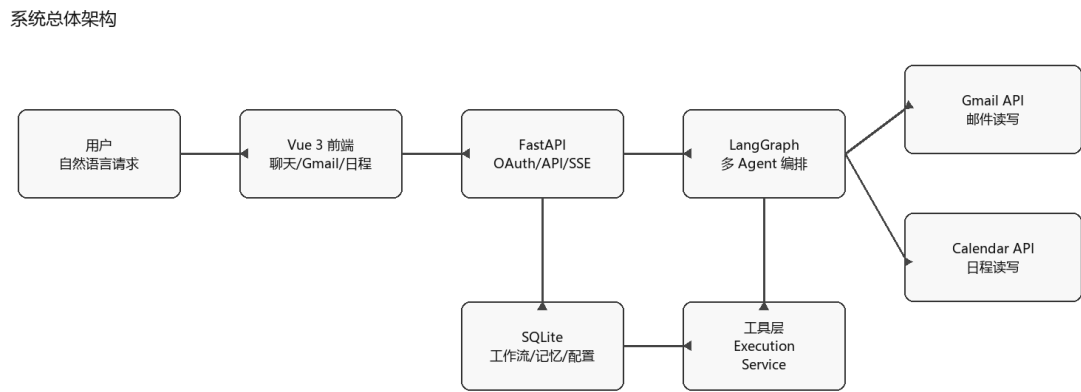
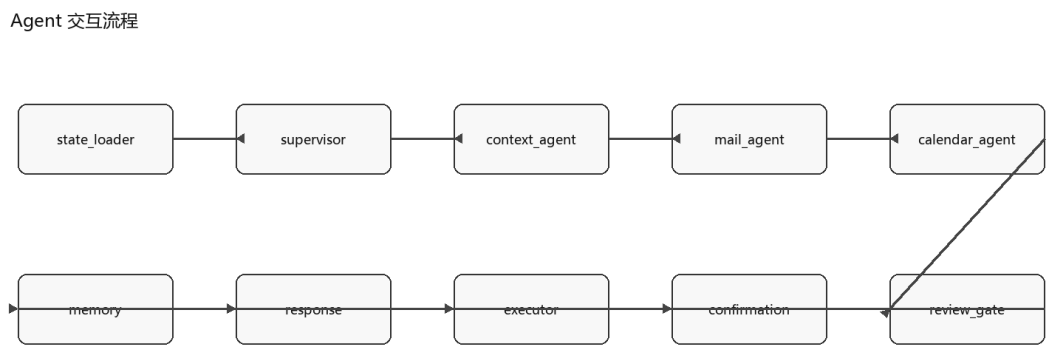


图 1 系统总体架构：前端、FastAPI、LangGraph、工具层、SQLite 与 Google 服务

3.2 Agent 交互流程

Agent 流程不是单个模型一次性回答，而是多个角色分工协作。Supervisor 先识别意图，Context Agent 处理上传文件或选中上下文，Mail Agent 与 Calendar Agent 分别处理专业任务，Review Gate 和 Confirmation Gate 负责关键字段与安全边界，Executor 只在确认后执行真实外部写操作。



写操作必须先形成草稿和 Proposal，用户确认后才进入 executor。

图 2 Agent 交互流程：从状态加载到确认执行和记忆抽取

3.3 数据流设计

会话状态、业务事实和外部执行状态分开保存。**LangGraph checkpoint** 保存对话和图运行状态；**app.sqlite3** 保存应用业务数据；**Markdown** 仅用于 README、项目文档、**Prompt** 模板和可读导出，不作为授权、执行或凭据的事实来源。

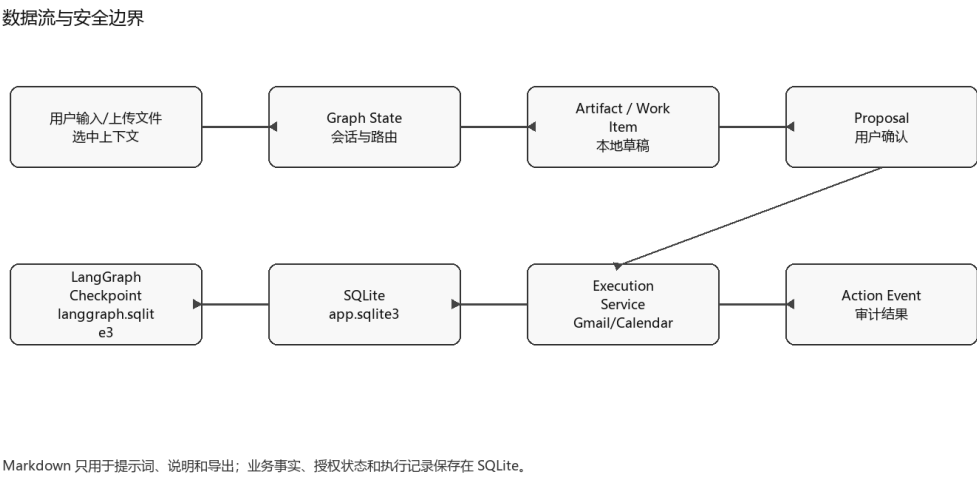


图 3 数据流设计：输入、草稿、Proposal、执行和审计记录的边界

四、关键实现与代码展示

4.1 Agent 核心循环

核心图在 `build_assistant_graph` 中显式声明节点顺序，每个节点通过 `timed_node` 包装以便记录耗时。这个结构让系统可以定位任务卡在哪个阶段，也方便测试导出 Mermaid 图。

```
builder.add_edge(START, "state_loader")
builder.add_edge("state_loader", "supervisor_agent")
builder.add_edge("supervisor_agent", "context_agent")
builder.add_edge("context_agent", "mail_agent")
builder.add_edge("mail_agent", "calendar_agent")
builder.add_edge("calendar_agent", "review_gate")
builder.add_edge("review_gate", "confirmation_gate")
builder.add_edge("confirmation_gate", "executor")
builder.add_edge("executor", "response_agent")
builder.add_edge("response_agent", "save_turn")
builder.add_edge("save_turn", "memory_extractor")
```

代码片段 1 LangGraph 主流程边定义

4.2 工具定义与职责隔离

工具函数通过上下文注入获得当前用户、会话、Gmail 客户端、Calendar 客户端和数据库 Session。不同 Agent 只拿到自己职责内的工具，避免邮件 Agent 调用日程执行工具，也避免 Response Agent 直接执行写操作。

```
mail_tools = {t.name for t in create_tools_for_agent("mail_agent")}
calendar_tools = {t.name for t in create_tools_for_agent("calendar_agent")}
context_tools = {t.name for t in create_tools_for_agent("context_agent")}

assert "create_email_draft" in mail_tools
assert "execute_calendar_event_draft" not in mail_tools
assert "create_calendar_event_draft" in calendar_tools
assert "send_email_draft" not in calendar_tools
assert {"search_emails", "read_email"}.issubset(context_tools)
```

代码片段 2 工具按 Agent 作用域隔离的测试

4.3 配置文件与 OAuth

项目支持 `.env` 和 `google_oauth.json` 两种本地配置方式。`google_oauth.json` 来自 Google Cloud Console，包含 `client_secret`，因此只允许本地私有使用，不能提交到 GitHub。README 中的回调地址与代码默认值保持一致。

```
GOOGLE_CLIENT_ID=你的-client-id
GOOGLE_CLIENT_SECRET=你的-client-secret
GOOGLE_REDIRECT_URI=http://localhost:8000/gmail/auth/callback

# 或将 Google Cloud Console 下载的 OAuth JSON
# 放在仓库根目录并命名为 google_oauth.json
```

4.4 AI IDE 使用

开发过程中使用 Trae CN / Codex 辅助完成需求拆解、代码阅读、README 与 .gitignore 检查、文档整理、测试命令确认和报告生成。AI IDE 主要承担“阅读与校对”角色，关键凭据配置、Google 授权、邮件发送和日程创建仍由本地应用和用户确认完成。

使用环节	具体作用
需求对齐	根据课程要求检查 README、文档结构、提交内容和报告章节是否覆盖评分点。
代码理解	辅助定位 LangGraph 主图、工具函数、OAuth 配置、SQLite 运行目录和前端页面。
安全检查	核对 .gitignore、.env.example、google_oauth.json、data/、.agents/ 等文件是否适合提交。
报告整理	根据真实功能截图补充测试与评估章节，解释截图之间的功能对应关系。
验证执行	运行 pytest、ruff 和 npm build，确认报告中的测试结果来自真实命令输出。

4.5 关键实现对应文件

为了便于评审快速定位代码，下面列出报告中关键设计与仓库实现之间的对应关系。

实现点	主要文件	说明
Graph 编排	app/graph/builder.py, multi_nodes.py	定义主流程节点、Agent 路由、确认门和执行节点。
工具调用	app/graph/tools.py	封装邮件、日程、联系人、记忆等工具，并做 Agent 作用域隔离。
OAuth 与配置	app/core/config.py, services/oauth.py	读取 .env 或 google_oauth.json，处理 Google 授权和 token。
邮件服务	services/gmail.py, api/gmail.py	解析 Gmail message、构造 RFC 822、执行草稿发送。
日程服务	services/calendar.py, api/calendar.py	解析时间、检查忙闲、写入 Google Calendar 事件。
前端交互	src/frontend/src/components/*	聊天、确认卡片、Gmail 页面、Calendar 页面和设置页。

五、测试与评估

5.1 自动化测试

自动化测试覆盖后端 Agent 工具、Assistant Graph、Gmail/Calendar 服务、文件解析、工作流、记忆、健康检查和日志脱敏等模块。前端通过 vue-tsc 与 Vite build 验证类型和生产构建。

测试项	命令	结果
后端单元测试	cd src/backend && ../../.venv/Scripts/python -m pytest	70 passed, 1 warning
后端静态检查	cd src/backend && ../../.venv/Scripts/python -m ruff check .	All checks passed
前端生产构建	cd src/frontend && npm run build	vue-tsc 与 Vite build 通过

5.2 Demo 截图与功能验证

本节截图来自本地实际运行的系统。截图不是孤立展示，而是按功能链路解释：应用内生成草稿和确认按钮证明 Agent 侧流程，真实邮箱或日历中的结果证明外部系统写入成功。

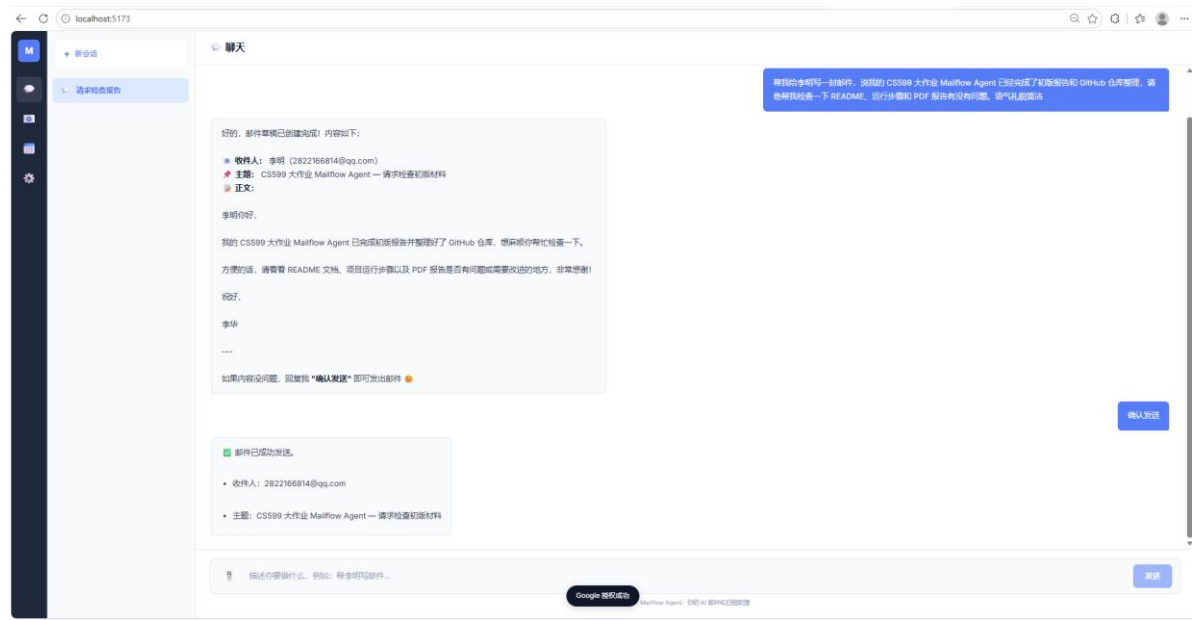


图 4 邮件功能：应用内生成邮件草稿，用户确认后显示发送成功



图 5 邮件功能：QQ 邮箱中真实收到该邮件，与图 4 构成发送结果验证

图 4 和图 5 是一组对应关系：图 4 说明 Mailflow Agent 根据自然语言生成邮件草稿，并通过确认按钮触发发送；图 5 则展示同一主题邮件已经进入真实邮箱，证明系统完成了从 Agent 草稿到 Gmail 发送再到收件端可见的闭环。

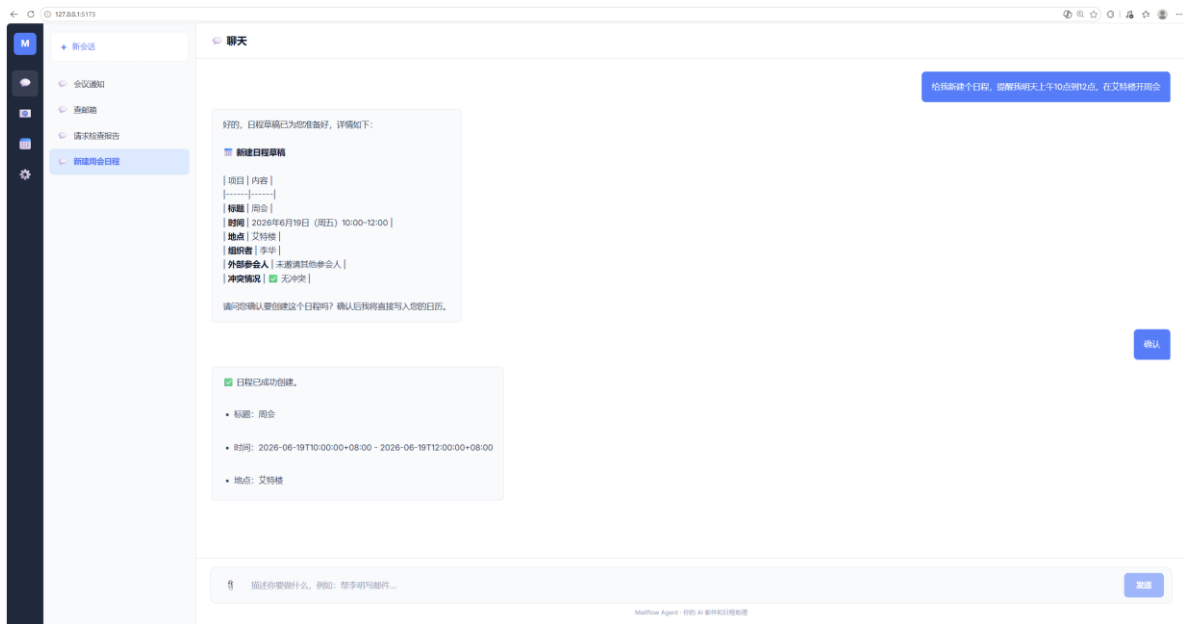


图 6 日程功能：应用内生成日程草稿，确认后返回创建成功结果

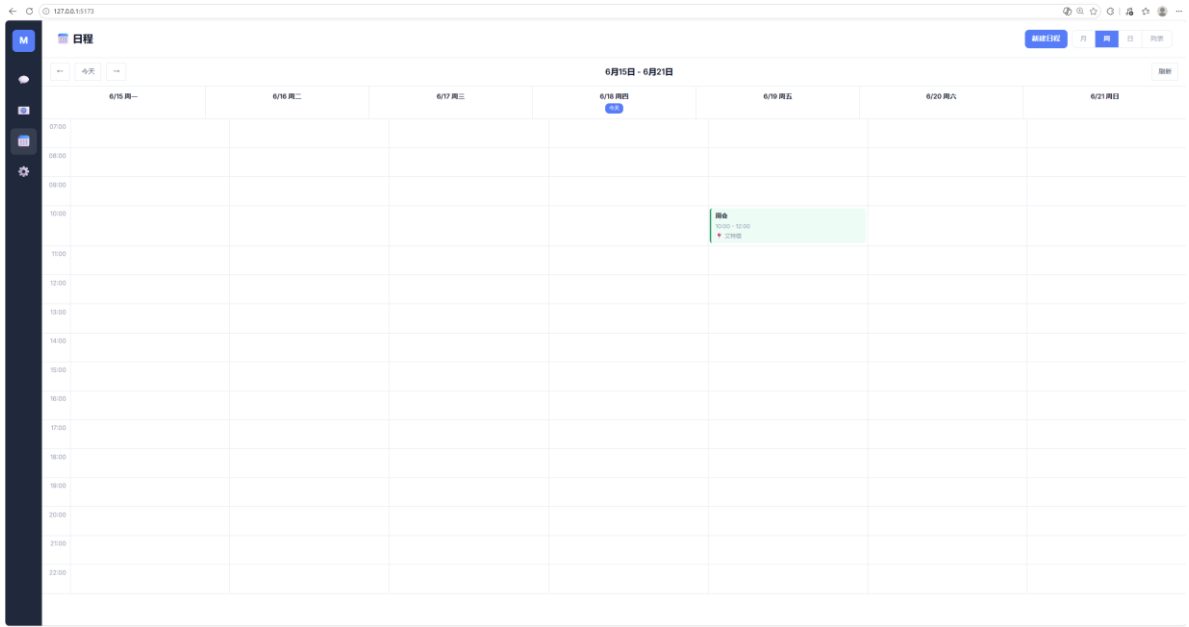


图 7 日程功能：周视图中出现对应日程，证明 Calendar 写入成功

图 6 和图 7 是日程链路的对应验证：前者体现 Agent 解析自然语言时间、地点和标题并等待确认，后者体现日程已经出现在系统日历周视图中。

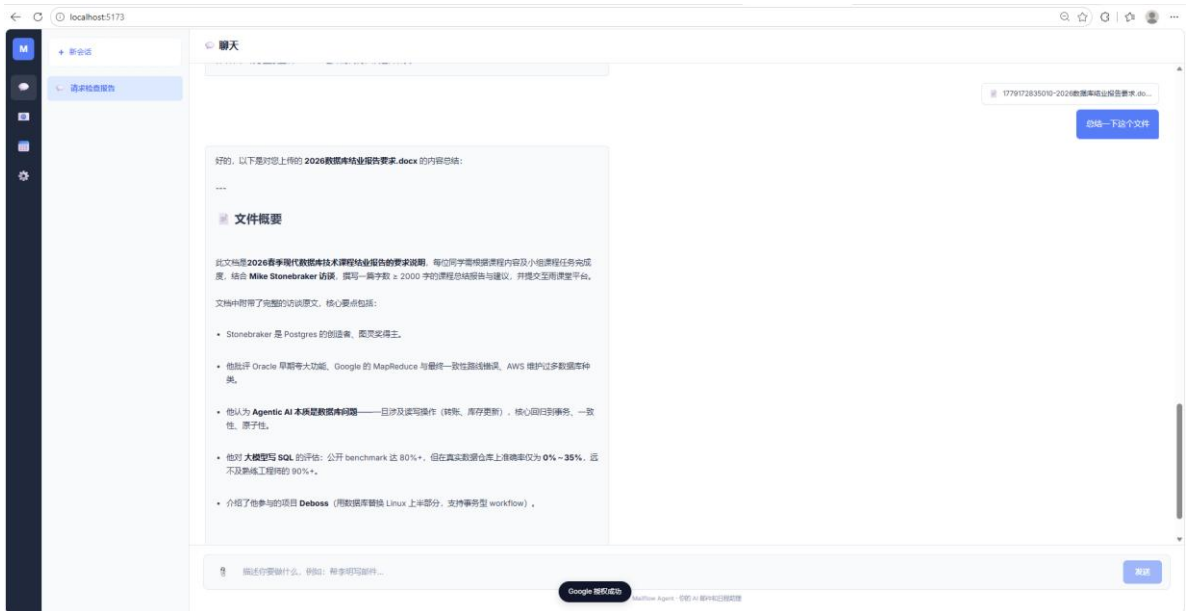


图 8 文件上下文功能：上传 Word 文档后，Agent 能读取并总结文件内容

文件截图证明系统不仅能处理邮件和日程，也能把用户上传的 Word 文档作为上下文。该能力可用于总结课程材料、辅助写邮件、准备日程说明，但文件内容仍按项目安全规则视为不可信上下文，不能替代用户确认。

5.3 Agent 行为评估

评估维度	观察结果
任务理解	能区分写邮件、查邮箱、创建日程、总结上传文件等不同意图。
字段补全	能从联系人、用户输入和默认设置中补全收件人、署名、时间、地点等字段。

安全确认	发送邮件和创建日程前会展示草稿，等待用户确认后才执行。
外部验证	邮件和日程均有真实外部系统结果截图，避免只停留在文本回复。
异常边界	项目文档定义了字段缺失、冲突、授权失败、文件不可读等处理规则。

5.4 Demo 结果小结

从截图和命令结果看，当前 MVP 已经覆盖课程项目所需的核心演示链路：自然语言输入、Agent 草稿生成、用户确认、外部服务写入、日历或邮箱结果验证、文件上下文读取，以及自动化测试通过。

- 邮件链路验证重点是“应用内确认发送”和“真实邮箱收到邮件”的对应关系。
- 日程链路验证重点是“Agent 生成草稿并执行”和“周视图出现事件”的对应关系。
- 文件链路验证重点是上传 Word 后能够提取内容并形成摘要，而不是只保存文件名。
- 测试链路验证重点是后端单元测试、静态检查和前端构建均可在本地环境复现。

5.5 评估结论

综合自动化测试和手工 Demo，当前系统已经达到课程 MVP 交付状态。仍需注意的是，本项目定位为本地个人助理，暂未加入多用户隔离、企业部署、OCR 和完整端到端自动化测试；这些内容已放入下一章的升级计划中。

六、系统升级与扩展

6.1 可扩展架构

当前系统已经把用户界面、API 路由、Agent 编排、工具函数和数据持久化分层。后续扩展新工具时，可以新增服务封装、Prompt、工具函数和前端页面，再接入 Supervisor 路由与 Proposal + Authorization 闭环，而不需要重写整个应用。

6.2 下一阶段计划

阶段	目标	说明
短期	补充端到端测试	为邮件发送、日程创建、OAuth 回调和文件上传增加更完整的自动化流程验证。
短期	优化 UI 证据链	把 Proposal 卡片、执行记录和外部验证结果展示得更清晰，便于 Demo 和排错。
中期	建立 Agent Benchmark	固定一组邮件、日程、上下文、安全拒绝任务，用于比较 Prompt 和模型版本。
中期	增强可观测性	记录节点耗时、工具调用结果、失败类型和可脱敏的调试摘要。
长期	多用户与部署	引入更严格的用户隔离、Docker 化、数据库迁移和云端部署策略。

6.3 AI 能力演进路径

- 从单轮生成走向多 Agent 协同：规划、上下文、专业工具、审查、确认和记忆分别由不同节点负责。
- 从 Prompt 经验走向可评估策略：把成功用例、失败用例和安全边界转化为可回归的测试集。
- 从本地个人助理走向企业流程助手：保留 Proposal 与 Authorization 机制，扩展到 Drive、Slack、Notion 或企业 OA。
- 从结果可用走向过程可信：持续强化字段来源、执行审计、敏感信息脱敏和恢复机制。

七、课程总结

7.1 个人收获

通过 Mailflow Agent，我对 Agentic AI 的理解从“让模型回答问题”转向“让模型参与真实工程流程”。只有当模型输出能够被状态机接住、被工具层约束、被用户确认、被数据库记录时，它才真正成为软件系统的一部分。

7.2 工程思维转变

这个项目让我更重视边界设计。邮件发送和日程创建看起来只是两个按钮，但背后需要 OAuth、字段来源、联系人解析、草稿版本、fingerprint、确认授权、执行幂等和失败恢复。工程上的可靠性不是靠模型“聪明”保证，而是靠系统把不确定性限制在可控范围内。

7.3 对课程的建议

- 建议继续保留从选题、Specs、架构、代码到测试报告的完整交付要求，因为这能促使项目从 Demo 走向工程化。
- 建议提供几个优秀 SDD 和 Agent 评估样例，帮助学生更早理解规格文档和行为评估应该写到什么粒度。
- 建议增加 OAuth、API Key、GitHub 提交安全和 .gitignore 的实践讲解，这些内容对真实项目非常重要。